

Assignment 5 solution

1. 【考点：网络应用模型】关于客户/服务器（C/S）模型与对等（P2P）模型的网络通信特征，下列描述中错误的是（ ）

- A. 在 C/S 模型中，服务器通常拥有固定的 IP 地址和熟知的端口号，且系统性能的瓶颈往往在于服务器的带宽和处理能力
- B. 在 P2P 模型中，随着参与节点数量（Peer）的急剧增加，每个节点下载完整文件的平均时间不仅不会显著增加，系统的整体共享能力反而会增强
- C. BitTorrent（BT）协议是一种纯粹的无结构 P2P 协议，在整个文件分发过程中绝对不需要任何集中式服务器的参与
- D. C/S 模型的客户端之间通常不能直接进行通信，必须通过服务器进行数据中转或指令调度

【答案】： C

【解析】：

A、D 正确：这是 C/S 模型的基本特征

B 正确：P2P 模型的最大优势在于可扩展性（Scalability）。新节点的加入不仅增加了服务请求，同时也带来了服务能力（带宽），因此下载时间趋于平稳，整体吞吐量增加

C 错误（陷阱）：BitTorrent（BT）网络虽然是 P2P 架构，但它并非完全不需要服务器。BT 架构中通常需要一个追踪服务器（Tracker Server）来维护当前正在下载/提供该文件的对等方列表。节点需要先连接 Tracker 获取 Peers 列表后，才能进行 P2P 传输。因此并非“绝对不需要”

2. 【考点：DNS 系统】在 DNS 系统的资源记录（Resource Record, RR）中，包含了（Name, Value, Type, TTL）四个字段。假设一台名为 `mail.nju.edu.cn` 的物理主机既作为该学校的主页 Web 服务器，又作为邮件服务器负责接收外部发给 `@nju.edu.cn` 的电子邮件。在 `nju.edu.cn` 域的授权域名服务器中，为确保这两项服务正常运行，最合理的 DNS 记录配置是（ ）

- A. 必须配置一条 Type 为 A 的记录将 `mail.nju.edu.cn` 映射到其 IP，且必须配置一条 Type 为 MX 的记录将 `nju.edu.cn` 映射到 `mail.nju.edu.cn`
- B. 只需要配置一条 Type 为 CNAME 的记录，即可同时实现 Web 和邮件的别名解析
- C. 必须配置一条 Type 为 NS 的记录，将 `nju.edu.cn` 的所有流量引向该主机的 IP 地址
- D. 配置一条 Type 为 MX 的记录，其 Name 字段直接填入 `mail.nju.edu.cn`，Value 字段填入该主机的 IP 地址即可

【答案】： A

【解析】：

Type A（Address）：实现主机名（如 `mail.nju.edu.cn`）到 IP 地址的映射。提供 Web 服务必须依赖 A 记录定位主机

Type MX（Mail Exchange）：用于邮件系统。其 Name 必须是邮件地址中的域名部分（如 `nju.edu.cn`），Value 是处理该域邮件的邮件服务器主机名（如 `mail.nju.edu.cn`）

因此，邮件发送方查找到 MX 记录得到 `mail.nju.edu.cn` 后，还需要进一步查询该主机名的 A 记录以获取 IP。A 选项完全正确。D 选项错误，MX 记录的 Value 只能是主机名，不能直接是 IP

3. 【考点：DNS 系统】假设某主机的本地域名服务器（Local DNS）无任何 DNS 缓存。当该主机向本地域名服务器发起对 `www.tsinghua.edu.cn` 的 DNS 查询时，若本地域名服务器至根域名服务器、顶级域名服务器、以及授权域名服务器均采用迭代查询模式。在最坏情况下（所有上级服务器均无相关缓存），本地域名服务器在这个过程中，作为客户端角色一共发出了多少个 DNS 查询请求报文？（ ）

- A. 1 个
- B. 2 个
- C. 3 个
- D. 4 个

【答案】：C

【解析】：

主机 → 本地域名服务器（递归查询，不算本地 DNS 发出的）

本地域名服务器 → 根域名服务器（发出第 1 个查询报文），根返回 .cn 顶级域名服务器的 IP

本地域名服务器 → .cn 顶级域名服务器（发出第 2 个查询报文），顶级返回 edu.cn 授权域名服务器的 IP

本地域名服务器 → tsinghua.edu.cn 授权域名服务器（发出第 3 个查询报文），授权返回

www.tsinghua.edu.cn 的终极 IP 地址

共发出 3 个查询报文

4. 【考点：FTP 协议】FTP 协议在进行文件传输时使用两个并行的 TCP 连接：控制连接和数据连接。下列关于这两种连接的底层机制和状态描述，正确的是（ ）

- A. 控制连接和数据连接都具有 Keepalive 机制，只要 FTP 客户端不主动退出，这两种连接就会在整个会话期间一直保持打开状态
- B. 在被动模式（PASV）下，FTP 服务器的数据连接必须绑定在固定的 20 端口上等待客户端连接
- C. FTP 控制连接是“带外（Out-of-band）”传输的，该连接具有状态特征，服务器必须在此连接上维护当前用户的当前工作目录等会话状态
- D. 为了提高传输效率，FTP 的控制报文（如登录用户名、密码、LIST 命令）和实际的文件数据默认都会经过应用层的 Base64 压缩编码后再通过 TCP 发送

【答案】：C

【解析】：

A 错：数据连接是非持久的。默认情况下，每传输一个文件或执行一次目录列表（LIST），就会动态建立一次新的数据连接，传输完毕后立刻关闭。只有控制连接是贯穿整个会话持续打开的

B 错：在被动模式（PASV）下，服务器开启一个随机的临时高位端口，并在控制连接中将该端口号告知客户端，由客户端主动向服务器的这个随机端口发起数据连接。只有主动模式（PORT）服务器才会主动使用 20 端口

C 正确：控制连接传输命令，数据连接传输文件内容，这种分离被称为“带外”控制。由于控制连接在整个会话中存在，FTP 是一种有状态（Stateful）协议，服务器需要记住用户的工作目录等

D 错：FTP 控制命令是明文的 7 位 NVT ASCII 码传输，没有强制的 Base64 压缩编码要求

5. 【考点：电子邮件系统】用户 A 使用 PC 上的 Outlook 客户端向用户 B 发送了一封带有高清 PDF 附件的电子邮件；用户 B 收到通知后，使用手机上的 Web 浏览器登录网页版邮箱查阅了该邮件及其附件。在此次完整的通信交互中，绝不会使用到的协议及动作组合是（ ）

- A. 发件方 Outlook 客户端使用 SMTP 协议将邮件推送到发件方邮件服务器
- B. 发件方邮件服务器使用 SMTP 协议将邮件推送到收件方邮件服务器
- C. 邮件中的 PDF 附件数据在通过 SMTP 传输前，被邮件客户端使用 MIME 机制转换为 ASCII 码文本
- D. 用户 B 的 Web 浏览器使用 POP3 协议从收件方邮件服务器拉取邮件内容以供网页显示

【答案】：D

【解析】：

A、B 正确：发信人 → 发件服务器，发件服务器 → 收件服务器，这两个“推送”阶段都使用 SMTP 协议

C 正确：SMTP 只能传输 7 位 ASCII 码。非文本附件（PDF）必须通过 MIME（如 Base64 编码）转换为 ASCII 格式后才能封装在 SMTP 报文中传输

D 错误：用户 B 使用的是 Web 浏览器（网页版邮箱）。浏览器与邮件服务器（通常是由该邮件服务商提供的 Web 接口服务器）之间的通信使用的是 HTTP 协议，而不是 POP3 或 IMAP。

POP3/IMAP 是专属邮件客户端（如 Foxmail、Outlook）与邮件服务器拉取邮件时使用的

6. 【考点：WWW 与 HTTP 协议】假设客户端浏览器需要请求一个包含 3 张嵌入式小图片（均位于同一台服务器）的 HTML 页面。忽略 DNS 解析时间、处理时延以及极小数据的传输时延，设客户端与服务器之间的往返传播时间恒定为 1 RTT。为了严格对比底层机制，规定客户端不采用多线程并行打开 TCP 连接。若分别采用以下三种 HTTP 协议版本/模式，从开始建立第一个 TCP 连接，到最终接收完所有图片数据，所需的总时间（以 RTT 计）由小到大排列正确的是（ ）

- ① HTTP/1.0（非持续连接）
- ② HTTP/1.1（持续连接，不采用流水线）
- ③ HTTP/1.1（持续连接，采用流水线 Pipelining）

- A. ③ < ② < ①，分别为 3 RTT, 5 RTT, 8 RTT
- B. ③ < ② < ①，分别为 4 RTT, 6 RTT, 8 RTT
- C. ③ < ② < ①，分别为 3 RTT, 5 RTT, 10 RTT
- D. ③ = ② < ①，分别为 4 RTT, 4 RTT, 8 RTT

【答案】：A

【解析】：

- ① HTTP/1.0（非持续连接）：每个对象都需要全新的 TCP 三次握手
HTML 页面：TCP 建立 (1 RTT) + HTTP GET/响应 (1 RTT) = 2 RTT
3 张图片：每次串行新建连接并请求， $3 \times 2 \text{ RTT} = 6 \text{ RTT}$ 。总计 = 8 RTT
- ② HTTP/1.1（持续连接，无流水线）：TCP 连接保持，但必须等前一个响应收到才能发下一个请求
HTML 页面：建立 TCP (1 RTT) + 获取 HTML (1 RTT) = 2 RTT
3 张图片：复用连接，连续发出 3 个请求（串行等待）， $3 \times 1 \text{ RTT} = 3 \text{ RTT}$ 。总计 = 5 RTT
- ③ HTTP/1.1（持续连接，有流水线）：收到 HTML 后，连续背靠背发出 3 个图片的 GET 请求
HTML 页面：建立 TCP (1 RTT) + 获取 HTML (1 RTT) = 2 RTT
3 张图片：同时发出 3 个 GET，服务器连续回复 3 个响应，这批数据的往返视为 1 RTT。总计 = 3 RTT

7. 【考点：WWW 与 HTTP 协议】为了降低服务器压力，HTTP 协议大量利用了缓存机制。当用户在浏览器中刷新一个以前访问过的网页时，浏览器向服务器发送的 HTTP GET 请求报文中包含了一个特定的首部字段（如 `If-Modified-Since`）。如果服务器验证发现该页面自上次缓存以来并未发生任何修改，服务器响应的行为及状态码应为（ ）

- A. 返回状态码 200 OK，但在响应体中不包含任何页面数据以节省带宽
- B. 返回状态码 301 Moved Permanently，告知浏览器资源未修改并直接读取本地文件
- C. 返回状态码 304 Not Modified，且响应报文不包含实体主体（Entity Body），指示浏览器直接使用本地缓存
- D. 返回状态码 403 Forbidden，拒绝提供重复数据，强制客户端读取缓存

【答案】：C

【解析】：

- A 错：200 OK 意味着请求完全成功，且按照标准，其响应体中必定包含被请求的资源实体（除非是 HEAD 请求）
- B 错：301 是永久重定向，用于网址变更
- C 正确：这是条件 GET 的标准行为。服务器收到 `If-Modified-Since`，检查发现未修改，返回 304 Not Modified，并且为了省流，响应体为空，浏览器收到后会自动从本地缓存加载页面
- D 错：403 是权限拒绝

8. 【考点：应用层与传输层的协议映射】在 TCP/IP 体系结构中，应用层协议为了实现其特定的网络功能，需要选择合适的传输层服务。下列应用层协议与其默认使用的传输层协议及熟知端口号的映射关系中，完全正确的一组是（ ）

- A. DNS: UDP 53 端口；HTTP: TCP 80 端口；POP3: UDP 110 端口
- B. DNS: UDP 53 端口；SMTP: TCP 25 端口；FTP 控制连接: TCP 21 端口
- C. HTTP: TCP 80 端口；SMTP: UDP 25 端口；FTP 数据连接: TCP 20 端口
- D. DNS: TCP 53 端口；POP3: TCP 110 端口；HTTP: UDP 80 端口

【答案】：B

【解析】：

DNS 绝大多数常规查询使用 UDP 53（区域传送使用 TCP 53）

HTTP 使用 TCP 80

FTP 使用 TCP 21（控制连接）和 TCP 20（主动模式数据连接）

SMTP（发送邮件）使用 TCP 25

POP3（接收邮件）使用 TCP 110

A 错在 POP3 必须是 TCP；C 错在 SMTP 必须是 TCP（邮件要求极高可靠性）；D 错在 HTTP 是 TCP，且 DNS 默认查询用 UDP

9. 【考点：WWW 与 HTTP 协议】HTTP 协议本身是一种无状态（Stateless）协议。为了在诸如电商网站中实现“购物车”功能，Web 系统广泛使用了 Cookie 技术来维持用户的会话状态。关于 Cookie 的工作原理，下列说法错误的是（ ）

A. Cookie 数据本质上是由 Web 服务器生成的，并通过 HTTP 响应报文中的 `Set-Cookie` 首部字段发送给浏览器

B. 浏览器接收到 Cookie 后，将其保存在客户端本地（内存或硬盘中）

C. 当浏览器再次向同一个服务器发起 HTTP 请求时，浏览器会自动在请求体（Body）中附带之前保存的 Cookie 数据，以表明自己的身份

D. 虽然 Cookie 解决了状态维护问题，但如果被恶意截获，可能会导致用户的会话（Session）被劫持

【答案】：C

【解析】：

C 选项错误：当浏览器后续向服务器发送请求时，Cookie 数据是封装在 HTTP 请求报文的 HTTP 首部行（Header 字段：`Cookie: ...`）中的，而不是放在请求体（Body）中。GET 请求通常没有 Body，但可以携带 Cookie

10. 【考点：电子邮件】在标准的因特网电子邮件系统中，包含了“用户代理（User Agent）”、“邮件服务器（Mail Server）”以及相关协议。假设用户甲的邮箱是 `jia@qq.com`，用户乙的邮箱是 `yi@163.com`。当用户甲向用户乙成功发送并最终被乙读取一封邮件的过程中，下列描述最准确反映组件交互关系的是（ ）

A. 甲的用户代理直接通过 SMTP 协议将邮件发送到乙的 163 邮件服务器，无需 QQ 邮件服务器介入

B. 甲所在的 QQ 邮件服务器必须运行 SMTP 服务器程序，同时也必须运行 SMTP 客户程序

C. 当乙的用户代理启动并向 163 邮件服务器请求查阅邮件时，163 邮件服务器作为 POP3 客户程序运行，乙的用户代理作为 POP3 服务器程序运行

D. 如果甲发送邮件时乙的计算机处于关机状态，该邮件将在甲的 QQ 邮件服务器中无限期滞留，直到乙开机

【答案】：B

【解析】：

A 错：甲必须先发给自己的代理服务器（QQ 服务器），QQ 服务器再发给 163 服务器

B 正确：QQ 邮件服务器在接收甲的用户代理发来的邮件时，充当 SMTP 服务器；随后它需要将邮件转发给 163 服务器时，又必须切换为 SMTP 客户角色发起连接

C 错：拉取邮件时，乙的用户代理是 POP3 客户，163 邮件服务器是 POP3 服务器

D 错：邮件会暂存在乙的 163 邮件服务器（收件方服务器）的邮箱队列中等待被拉取，而不是滞留在 QQ 邮件服务器

11. 【考点：DNS 系统】在一个大规模的 Kubernetes 微服务集群中，服务 A 需要高频调用外部支付接口 `api.payment.example.com`。为了防止大量的 DNS 查询打满核心交换机，运维团队在集群内部署了本地 DNS 服务器（Local DNS），并配置了严格的解析策略。假设该 Local DNS 缓存刚好失效，且采用“对客户端递归、对上游迭代”的标准解析模式。关于本次服务 A 解析该域名的完整网络交互过程，以下描述完全正确的是（ ）

- A. 服务 A 向 Local DNS 发送迭代查询请求，Local DNS 随后依次向根域名服务器、顶级域名服务器（TLD）、权威域名服务器发送递归查询请求
- B. 根域名服务器收到 Local DNS 的请求后，如果不知道目标 IP，会主动代替 Local DNS 向 `.com` 顶级域名服务器发起查询，并将最终结果层层返回
- C. 在迭代查询过程中，Local DNS 发送的请求报文的目的 IP 地址会不断变化（依次为根、TLD、权威 DNS 的 IP），但客户端（服务 A）只需耐心等待 Local DNS 的最终响应
- D. 为了加快解析速度，`.com` 顶级域名服务器（TLD）会直接在响应报文中封装 `api.payment.example.com` 的最终 IP 地址返回给 Local DNS

【答案】：C

【解析】：

- A 选项错误：策略说反了。标准模式是“客户端向 Local DNS 发起递归查询（你帮我查到底）”，“Local DNS 向上游发起迭代查询（不代劳，只指路）”
- B 选项错误：这是递归查询的特征。既然 Local DNS 对上游采用迭代查询，根域名服务器绝对不会“主动代替”它去查，而是直接返回 TLD 的 IP 地址，让 Local DNS 自己去请求 TLD
- D 选项错误：顶级域名服务器（TLD）通常只管理下一级权威域名的 NS 记录（如 `example.com` 的 DNS 服务器地址），它不知道具体主机（如 `api`）的 IP 地址。最终的 IP 是由 `example.com` 的权威域名服务器解析并返回的
- C 选项完全正确：服务 A 只需发一次递归请求并等待；而 Local DNS 作为“苦力”，需要自己构建并发送 3 次迭代查询报文，分别发往根、TLD 和权威 DNS，所以网络层目的 IP 在不断变化
12. 【考点：WWW与HTTP协议】某前端团队在开发一个复杂的大型监控 Dashboard 时，页面初始化需要向后端并发请求 50 个不同的 RESTful API 接口来获取图表数据
当前系统采用 HTTP/1.1 协议 且默认启用了 `Connection: keep-alive`。在没有配置任何代理及 HTTP/2 升级的情况下，关于客户端与服务端的网络传输特性，以下分析错误的是（ ）
- A. 启用 `keep-alive` 后，多个 HTTP 请求可以复用同一个底层的 TCP 连接，从而显著减少了频繁三次握手和四次挥手带来的网络延迟开销
- B. HTTP 协议本身是无状态的（Stateless），这 50 个请求在协议层面上互相独立。如果需要识别用户身份，必须依赖 Cookie、Token 等额外的应用层头部字段
- C. 虽然 HTTP/1.1 支持流水线（Pipelining）技术，允许客户端一次性连续发送多个请求，但服务端依然必须严格按照请求到达的顺序逐一返回响应
- D. HTTP/1.1 的流水线（Pipelining）技术已经从根本上彻底解决了“队头阻塞（Head-of-Line Blocking）”问题，因此这 50 个请求在单个 TCP 连接中不会互相影响

【答案】：D

【解析】：

- A、B 选项完全正确：这是 HTTP/1.1 长连接的基础定义以及 HTTP 无状态特性的标准描述
- C 选项正确：这是流水线（Pipelining）的硬性规定。客户端可以一口气发请求 1、2、3，但服务端即使先处理完了 2 和 3，也必须等 1 处理完并发送后，才能发送 2 和 3
- D 选项错误：这恰恰是 HTTP/1.1 最著名的架构缺陷。正因为“必须按序响应”（如选项 C 所述），一旦第一个请求（队头）在服务端处理极慢或者报文丢失，后续所有已经处理完的响应都必须在缓冲区排队等待。这种现象正是 HTTP 层的队头阻塞。直到引入 HTTP/2 的多路复用（Multiplexing，基于二进制分帧），才真正在单个 TCP 连接上解决了这个问题
13. 【考点：FTP协议】某企业的数据库每日定时进行全量备份，生成的压缩包需要通过 FTP 协议上传至远端的异地灾备中心
异地灾备中心出于极其严格的安全合规要求，其边界防火墙配置如下：仅允许外部 IP 访问灾备中心的 TCP 21 端口，所有访问其他任何端口的入站（Inbound）流量均被强制丢弃；但允许灾备中心向外发起任何出站（Outbound）流量
在此网络约束下，为了确保备份文件能够成功上传，FTP 客户端和服务端应该如何协商建立连接？（ ）

- A. FTP 客户端必须配置为主动模式 (Active/PORT)。客户端通过 21 端口告知服务端自己的数据监听端口, 由灾备中心服务端 (20 端口) 主动向客户端发起建立数据连接
- B. FTP 客户端必须配置为被动模式 (Passive/PASV)。灾备中心服务端在 21 端口协商后, 开放一个随机高端口, 客户端主动连接该高端口进行数据传输
- C. 无论主动模式还是被动模式都无法成功传输数据, 必须在防火墙上额外开放 TCP 20 端口的入站权限
- D. 客户端与服务端通过 21 端口建立控制连接后, 直接复用该 TCP 21 连接进行二进制数据的透传, 无需建立额外的数据连接

【答案】：A

【解析】：

- D 选项错误: FTP 的核心原理是“控制平面与数据平面分离”(带外控制)。控制连接 (21) 只传命令, 数据连接 (20 或随机端口) 传文件, 绝不复用
- B 选项错误: 如果采用被动模式, 服务端会开启一个随机的高端口 (如 50000), 等待客户端来连。但是, 题干明确指出“防火墙丢弃除 21 以外的所有入站流量”, 客户端根本连不上服务端的这个高端口, 数据连接建立失败
- C 选项错误: 不需要开放 20 端口的入站权限
- A 选项正确: 在主动模式下, 客户端通过 21 端口发命令: “服务端你好, 我开启了本地的 60000 端口, 你来连我传数据”。服务端收到后, 从自己的 20 端口发起连接, 目标是客户端的 60000 端口。由于防火墙“允许任何出站流量”, 这个由服务端向外发起的 TCP SYN 报文可以顺利穿透防火墙, 数据连接成功建立

14. 【考点: 电子邮件】在高可用分布式系统中, 当某个节点发生 OOM (内存溢出) 崩溃时, 监控 Agent 会自动抓取 5MB 的 Core Dump 核心转储文件 (纯二进制格式), 并通过系统内置的邮件发送程序 (User Agent) 将其作为附件发送给运维负责人的邮箱
- 运维负责人随后在自己的电脑上使用 Outlook 客户端查收这封带有二进制附件的告警邮件。在这个完整的流转过程中, 关于涉及到的协议和格式转换, 以下说法正确的是 ()
- A. Agent 将 5MB 的二进制文件直接封装在 SMTP 报文的数据体中, 发送给企业的邮件发送服务器
 - B. 邮件在网络中传输时, 该二进制附件必须经过 MIME (多用途网际邮件扩充) 协议中的 Base64 等编码技术, 将其转换为 7 位 ASCII 码格式
 - C. 运维负责人的 Outlook 客户端使用了 SMTP 协议, 从他的邮件接收服务器中拉取 (Pull) 这封邮件到本地进行阅读
 - D. 由于邮件总大小超过了 1MB, POP3 协议无法处理这种大附件, Outlook 客户端必须使用 IMAP 协议来接收这封邮件

【答案】：B

【解析】：

- A 选项错误: 极其经典的基础限制。原生 SMTP 协议只能处理纯文本 (即 7 位 ASCII 码)。它完全不认识也不支持直接传输二进制数据 (如图片、视频、Core Dump 文件等)
 - C 选项错误: SMTP 是一个“推 (Push)”协议, 用于发送方 UA 到发送方服务器、以及发送方服务器到接收方服务器。接收方 UA 去服务器上下载 (拉取) 邮件时, 使用的是 POP3 或 IMAP 协议, 绝不可能用 SMTP 接收邮件
 - D 选项错误: POP3 完全可以处理带有大附件的邮件 (只要服务器配置允许), 它与文件大小无关。IMAP 和 POP3 的主要区别在于 IMAP 提供了在服务器端管理邮件 (分类、按需下载附件) 的联机功能, 而 POP3 倾向于全部下载到本地
 - B 选项正确: 正因为 SMTP 只能传 7 位 ASCII, 所以诞生了 MIME 协议。MIME 就像一个翻译官, 它将原本不能用 SMTP 传输的二进制附件, 用 Base64 等算法硬生生地转码成由一堆英文字母和数字组成的 7 位 ASCII 字符串, 这样 SMTP 就能正常传输了
15. 【考点: 网络应用模型】某云计算大厂需要向 10,000 台物理宿主机同时分发一个大小为 2GB 的核心组件 Docker 镜像
- 架构师对比了传统的 C/S 模型 (将镜像放在中心化的 HTTP 文件服务器上) 和 P2P 模型 (使用类似 BitTorrent 协议进行镜像播种)

假设中心服务器的上传带宽固定为 u_s ，每台宿主机的下载带宽为 d ，上传带宽为 u ，需要分发的总主机数为 N ，文件大小为 F 。以下论述严格正确的是（ ）

- A. 在 C/S 模型中，分发完成的最短时间取决于 $\max \{F/d, F/u_s\}$ ，即要么受限于单台客户端的下载极限，要么受限于服务器的上传极限，与主机数量 N 无关
- B. 在 P2P 模型中，由于每个节点在下载的同时也在上传，整个系统的总上传能力变为了 $u_s + N \times u$ 。因此，即使主机数 N 趋于无穷大，分发时间也会收敛于一个极小的常数
- C. 在 C/S 模型中，服务器必须串行或并发地向每个客户端发送完整的 2GB 数据，因此当 N 非常大时，完成全部分发的理论最短时间下限将直接正比于 $N \times F / u_s$
- D. P2P 模型之所以效率高，是因为它摒弃了传输层的 TCP 协议，直接在网络层通过 IP 组播 (Multicast) 技术实现了“一发多收”

【答案】： C

【解析】：

A 选项错误：C/S 模型的致命瓶颈在于服务器带宽。服务器必须向 N 个客户端各发送一次文件，总共要发送 $N \times F$ 的数据量。因此分发时间 $D_{C/S} \geq N \times F / u_s$ 。它极度依赖主机数量 N

B 选项错误：虽然 P2P 系统的总上传能力是 $u_s + N \times u$ ，但总需求也是 $N \times F$ 。当 N 趋于无穷大时，分发时间 $D_{P2P} \approx F / u$ （假设节点的上传带宽是瓶颈），它会逼近于某一个常量下限，而不是“极小的常数”或趋近于 0

D 选项错误：典型的 P2P（如 BitTorrent）依然建立在端到端的 TCP（或某些 UDP 变种）单播连接之上，而不是网络层的硬件组播

C 选项完全正确：这是 C/S 模型的经典理论公式： $D_{C/S} = \max \{N \cdot F / u_s, F / d_{\min}\}$ 。当客户端数量 N 达到万级别时，服务器带宽绝对成为瓶颈，此时所需时间直接随 N 呈线性爆炸式增长。这正是工程中为什么大规模镜像分发（如阿里 Dragonfly、Uber Kraken）必须采用 P2P 架构的核心理论支撑

16. 12. A

POP3 用于用户代理从邮件服务器读取邮件，说法 I 正确。用户代理向邮件服务器发送邮件使用的是 SMTP，而非 POP3。邮件服务器之间的邮件传输也由 SMTP 实现，与 POP3 无关。POP3 支持在一条 TCP 连接中收取多封邮件，无须为每封邮件单独建立连接，说法 IV 正确。

17. 19. D

浏览器不支持并行 TCP 连接，使用非持续的 HTTP/1.0 协议，因此每传输一个 Web 页和小图像文件都要建立一次 TCP 连接。第一次建立 TCP 连接时，前两次握手花 1RTT，第三次握手报文段中可以携带 HTTP 请求，服务器收到请求后返回 Web 页，共花 2RTT。之后，每传输一个图像文件都要花 2RTT。因此，到接收完所有内容，需要的总时间至少是 $2 \times 8 = 16RTT$ 。注意，若浏览器支持并行 TCP 连接，则请求 Web 页仍要花 2RTT，但收到 Web 页后，可建立 7 个并行的 TCP 连接请求图像文件，传输图像的过程仅花 2RTT，总时间为 4RTT。

18.

03. 【解答】

- 1) 以太网帧的数据部分是 IP 数据报, 只要数出相应字段所在的字节即可。由图 3 可知以太网帧首部有 $6+6+2=14\text{B}$, 由图 4 可知 IP 数据报首部的目的 IP 地址字段前有 $4\times 4=16\text{B}$, 从图 2 的帧第 1 字节开始数 $14+16=30\text{B}$, 得到目的 IP 地址为 40.aa.62.20 (十六进制), 转换成十进制为 64.170.98.32。由图 2 可知以太网帧的前 6 字节 00-21-27-21-51-ec 是目的 MAC 地址, 即为主机的默认网关 10.2.128.1 端口的 MAC 地址。
- 2) ARP 用于解决 IP 地址到 MAC 地址的映射问题。主机的 ARP 进程在本以太网以广播形式发送 ARP 请求分组, 在以太网上广播时, 以太网帧的目的地址为全 1, 即 FF-FF-FF-FF-FF-FF。
- 3) HTTP/1.1 协议以持续的非流水线方式工作时, 服务器发送响应后仍在一段时间内保持这段连接, 客户机在收到前一个请求的响应后才能发出下一个请求。注意题目说的是从发出 Web 请求开始, 所以此时 TCP 连接已经建立。第一个 RTT 用于请求 Web 页面, 客户机收到第一个请求的响应后 (还有五个请求未发送), 每访问一次对象就用去一个 RTT。因此共需 $1+5=6$ 个 RTT 后浏览器收到全部内容。
- 4) 私有地址和 Internet 上的主机通信时, 须由 NAT 路由器进行网络地址转换, 把 IP 数据报的源 IP 地址 (本题为私有地址 10.2.128.100) 转换为 NAT 路由器的一个全球 IP 地址 (本题为 101.12.123.15)。因此, 源 IP 地址字段 0a 02 80 64 变为 65 0c 7b 0f。IP 数据报每经过一个路由器, TTL 值就减 1, 并重新计算首部检验和。若 IP 分组的长度超过输出链路的 MTU, 则总长度字段、标志字段、片偏移字段也会发生变化。

19. 04. 【解答】

- 1) 从 t_0 到 t_1 期间, 除了 HTTP, H1 还运行了 DNS 应用层协议, 以将域名转换为 IP 地址。DNS 运行在 UDP 之上, UDP 将应用层交付的 DNS 报文添加首部后, 向下交付给 IP 层, IP 层使用 IP 数据报进行封装, 封装好后, 向下交付给数据链路层, 数据链路层使用 CSMA/CD 帧进行封装。因此, 逐层封装关系如下: DNS 报文→UDP 数据报→IP 数据报→CSMA/CD 帧。传统以太网在数据链路层采用 CSMA/CD 协议, 因此使用 CSMA/CD 帧进行封装。提示: 在数据链路层对该报文的封装解释为以太网 V2 帧 (或以太网帧) 会更合适, 标准答案给出的 CSMA/CD 帧相对而言并不算特别合适。CSMA/CD 协议更多地用在以传统集线器互连的以太网中。交换机可工作于全双工方式, 通常不采用 CSMA/CD 协议。
- 2) t_0 时刻, H1 的 ARP 表和 S 的交换表为空。H1 利用浏览器通过域名请求访问 Web 服务器。因为要先解析域名, 查询该域名对应的 IP 地址, 所以要先向本地域名服务器发送 DNS 查询报文。ARP 表为空, 因此需要先发送 ARP 请求分组, 查询本地域名服务器对应的 MAC 地址, 这个帧的目的 MAC 地址是 FF-FF-FF-FF-FF-FF。S 接收到这个帧, 在交换表中记录 MAC 地址为 00-11-22-33-44-cc, 位于端口 4, 然后广播该帧。当本地域名服务器收到 ARP 请求后, 向 H1 发送 ARP 响应分组。S 接收到这个帧, 在交换表中记录 MAC 地址为 00-11-22-33-44-bb, 位于端口 1, 然后将该帧从端口 4 发送出去。得到了域名对应的 IP 地址, 发现不在本局域网中, 需要通过路由表转发。H1 的 ARP 表中没有路由器对应的 MAC 地址, 因此需要先发送 ARP 请求分组, 查询路由器对应的 MAC 地址, 这个帧的目的 MAC 地址是 FF-FF-FF-FF-FF-FF。S 接收到这个帧, 广播该帧。当路由器收到 ARP 请求后, 向 H1 发送 ARP 响应分组。S 接收到这个帧, 在交换表中记录 MAC 地址为 00-11-22-33-44-aa, 位于端口 2, 然后将该帧从端口 4 发送出去。现在, H1 就能发送 HTTP 请求。在整个过程中, 并没有涉及 H2, H2 没有主动发送数据, 因此 S 不会记录 H2 的 MAC 地址和端口, t_1 时刻 S 的交换表如下所示。

MAC 地址	端 口
00-11-22-33-44-cc	4
00-11-22-33-44-bb	1
00-11-22-33-44-aa	2

- 3) 由步骤 2) 的分析可知, H2 至少会接收到 2 个和此次 Web 访问相关的帧。接收到的均是封装 ARP 查询报文的以太网帧; 这些帧的目的 MAC 地址均是 FF-FF-FF-FF-FF-FF。

04.【解答】

- 1) 在 FTP 会话期间, 控制连接一直处于保持状态, 是持久的。当每次需要传输文件时, FTP 客户和服务端之间会建立一个临时的数据连接, 用于传输文件数据, 是非持久的。控制连接用于传输命令和控制信息, 登录操作涉及身份验证、发送命令等控制信息, 因此 H 登录 FTP 服务器时建立的 TCP 连接是控制连接。
- 2) 建立连接时, FTP 客户发送的第一次握手 SYN 段要消耗一个序号, 选择的初始序号为 100, 因此发送文件 F 时, 第一个字节的序号为 101。文件 F 共有 18000 个字节, 需占用 18000 个序号, 释放连接时, FTP 客户发送的第一次挥手 FIN 段也要消耗一个序号, 所以该 FIN 段的序号为 18101, 因此 TCP 服务器发送的第二次挥手 ACK 段的确认序号是 18102。
- 3) 拥塞窗口在每个传输轮次后的变化如下表所示。前两个传输轮次, 拥塞窗口小于阈值, 拥塞窗口按指数增长; 第 2 个传输轮次结束后, 拥塞窗口增长到 4MSS, 此后每经过一个传输轮次, 拥塞窗口增加 1MSS。当 H 收到确认序号为 2101 的确认段时, 表示服务器已收到 2000B 数据, 即 2 个报文段, 此时还处在第 2 个传输轮次 (慢开始阶段), 拥塞窗口还未达到阈值, 发送方每收到一个确认, 拥塞窗口就加 1, 所以此时拥塞窗口为 $2 + 1 = 3\text{MSS}$ 。当 H 收到确认序号为 7101 的确认段时, 表示服务器已收到 7 个报文段, 第 3 轮传输结束后, 发送方共发送了 $1 + 2 + 4 = 7\text{MSS}$ 数据, 所以拥塞窗口大小为 5MSS。

N 个传输轮次后	初始时	$N = 1$	$N = 2$	$N = 3$
拥塞窗口大小	1MSS	2MSS	4MSS	5MSS

- 4) 每个传输轮次传输的数据量如下表所示, 文件 F 的大小 $18000\text{B} = 18\text{MSS}$, 则发送完 F 要经过 5 个 RTT, 此外还要 1 个额外的 RTT 用来建立 TCP 连接。因此, H 从请求建立数据连接开始, 到确认 F 已被服务器全部接收为止, 至少需要 $6\text{RTT} = 6 \times 10\text{ms} = 60\text{ms}$; 期间应用层数据平均发送速率是 $18000\text{B} \div 60\text{ms} = 300 \times 10^3 \text{B/s} = 0.3\text{MB/s} = 2.4\text{Mb/s}$ 。

k	$k = 1$	$k = 2$	$k = 3$	$k = 4$	$k = 5$
第 k 轮最大数据传输量	1MSS	2MSS	4MSS	5MSS	6MSS

21. 【考点: DNS 系统】假设主机 H1 (IP: 192.168.1.10/24) 的网络配置中, 默认网关为 192.168.1.1/24, 本地 DNS 服务器为 192.168.1.53/24。当前 H1 的 ARP 缓存表和 DNS 缓存表均为空。当用户在 H1 的浏览器中输入 [http://www.tsinghua.edu.cn] (http://www.tsinghua.edu.cn) 并按下回车后, 直到 H1 成功发出建立 HTTP 连接的 TCP SYN 报文为止, 主机 H1 的网卡一共发出了多少个 ARP 请求报文? (假设网络无丢包, DNS 采用递归查询)
- A. 1 个
B. 2 个
C. 3 个
D. 4 个

【答案】: B

【解析】:

H1 首先需要解析域名, 向本地 DNS (192.168.1.53) 发 UDP 查询

H1 计算发现本地 DNS 与自己在同一子网, 因此直接在局域网内广播发送第 1 个 ARP 请求, 获取 DNS 服务器的 MAC 地址

H1 收到 DNS 响应, 获得了清华大学 Web 服务器的公网 IP (假设为 166.111.4.100)

H1 准备发送 TCP SYN 给该公网 IP。计算发现目标 IP 与自己不在同一子网, 数据包必须交由网关 (192.168.1.1) 转发

H1 再次广播发送第 2 个 ARP 请求, 获取网关的 MAC 地址。随后才将封装有 TCP SYN 的以太网帧发给网关。因此共发出 2 个 ARP 请求

22. 【考点：FTP 协议】某企业内部局域网采用典型的 NAPT（网络地址端口转换）接入互联网。局域网内的主机 Client（私网 IP 为 10.0.0.5）尝试使用主动模式（PORT）的 FTP 客户端程序，从公网上的 FTP 服务器 Server（公网 IP 为 202.118.1.1）下载文件。假设该 NAPT 路由器是基础版本，不具备 ALG（应用层网关）功能。关于此过程的描述，正确的是（ ）

- A. 控制连接和数据连接都能成功建立，文件可正常下载
- B. 控制连接建立失败，因为 NAPT 会拦截目的端口为 21 的 TCP SYN 报文
- C. 控制连接建立成功，但数据连接建立失败，因为 Server 无法向 10.0.0.5 的临时端口发起 TCP 连接
- D. 控制连接建立成功，数据连接建立失败，因为 Client 无法向 Server 的 20 端口发起 TCP 连接

【答案】：C

【解析】：

控制连接：Client 主动向 Server 的 21 端口发起 TCP 连接，NAPT 能正常转换源 IP 和源端口，控制连接建立成功

数据连接（主动模式）：Client 会在控制连接中发送 PORT 命令（例如 PORT 10,0,0,5,195,84），告知 Server 自己的私网 IP 和监听的端口。由于 NAPT 仅修改 IP 首部和 TCP 首部，不会修改 TCP 载荷中的应用层数据，因此 Server 收到的是私网 IP 10.0.0.5。Server 随后会从自己的 20 端口主动向 10.0.0.5 发起 TCP SYN。由于 10.0.0.5 是不可路由的私有地址，该连接请求会在公网上被丢弃，导致数据连接建立失败。这也是为什么在 NAT 环境下通常推荐使用被动（PASV）模式

23. 【考点：电子邮件格式与 MIME】用户 A 准备通过电子邮件向用户 B 发送一张大小为 1.5 MB（即 1.5×10^6 字节）的 JPEG 格式高清图片附件。由于 SMTP 协议只能传输 7 位 ASCII 码，邮件客户端采用了标准的 MIME Base64 编码机制对该图片进行转换。假设不考虑 IP 和 TCP 首部开销，也不考虑邮件本身的文本内容及首部字段，仅考虑附件数据的传输，在带宽为 10 Mbps（即 10^7 bps）的理想链路上，发送方邮件服务器将该编码后的附件数据推送到接收方邮件服务器至少需要消耗多少传输时延？（ ）

- A. 1.2 秒
- B. 1.5 秒
- C. 1.6 秒
- D. 2.0 秒

【答案】：C

【解析】：

Base64 编码规则：将每 3 个字节（24 bits）的二进制数据转化为 4 个字节（32 bits）的可打印 ASCII 字符。因此，数据体积会扩大三分之一，即变为原来的 $4/3$ 倍

编码后大小： $1.5 \text{ MB} \times 4/3 = 2.0 \text{ MB} = 2.0 \times 10^6 \text{ Bytes}$

换算为比特： $2.0 \times 10^6 \text{ Bytes} \times 8 \text{ bits/Byte} = 1.6 \times 10^7 \text{ bits}$

传输时延计算： $t = \text{数据量} / \text{带宽} = 1.6 \times 10^7 \text{ bits} / 10^7 \text{ bps} = 1.6 \text{ 秒}$

24. 【考点：WWW 与 HTTP 协议】HTTP/1.1 协议默认开启持久连接（Persistent Connection，即 Connection: keep-alive）。假设客户端浏览器与 Web 服务器建立了一条 HTTP/1.1 持久连接，在连续请求并接收完 5 个小图片后，浏览器所在的本地主机突然因断电而异常关机。此时，Web 服务器端的 TCP 连接控制块（TCB）最先会处于什么状态？（ ）

- A. FIN_WAIT_1 状态
- B. ESTABLISHED 状态
- C. CLOSE_WAIT 状态
- D. TIME_WAIT 状态

【答案】：B

【解析】：

客户端异常断电关机，意味着操作系统直接崩溃，无法执行任何 TCP 四次挥手流程，即客户端不会向服务器发送 FIN 报文

从 Web 服务器的视角来看，它并不知道客户端已经崩溃，网络本身也没有发生断开。因此，服务器的该 TCP 连接依然保持在 ESTABLISHED（已建立）状态

服务器只有在后续尝试向客户端发数据超时，或者其 TCP 保活计时器（Keepalive Timer）超时（通常需要 2 小时）后发送探测报文未果，才会最终关闭该连接

25. 【考点：电子邮件系统的组成结构】在标准的 Internet 电子邮件体系中，包含了发送方用户代理、发送方邮件服务器、接收方邮件服务器和接收方用户代理。以下关于各组件之间通信协议及其底层传输层角色的描述，完全准确无歧义的是（ ）

- A. 接收方邮件服务器在向接收方用户代理投递邮件时，充当 POP3/IMAP 的客户端角色
- B. 发送方邮件服务器在向接收方邮件服务器转发邮件时，既作为 SMTP 服务器角色，又作为 SMTP 客户端角色
- C. 用户使用 Web 浏览器（网页版邮箱）撰写并发送邮件时，浏览器与邮件服务器之间使用 SMTP 协议
- D. 发送方邮件服务器在向接收方邮件服务器转发邮件时，充当的是 SMTP 客户端角色

【答案】：D

【解析】：

A 错：接收方用户代理由拉取邮件时（POP3/IMAP），用户代理是客户端，邮件服务器是服务器端

B 错：发送方邮件服务器在“转发”这一特定动作时，仅仅充当 SMTP 客户端。只有在接收用户代理的邮件时，它才充当 SMTP 服务器

C 错：使用 Web 浏览器（网页邮箱）时，浏览器与 Web 服务器之间使用的是 HTTP 协议，而不是 SMTP 协议。（服务器内部再调用 SMTP 引擎）

D 正确：发件服务器向收件服务器推送邮件，是发起 TCP 连接的一方，因此明确充当 SMTP 客户端

26. 【考点：网络应用模型】在现代基于 C/S 模型的 Web 应用中，HTTP 是无状态协议，通常依赖 Cookie 来维持会话状态。假设用户在办公室（PC 使用固定以太网）登录了某电商网站进行购物，其购物车信息保存在服务器端，同时服务器下发了一个带有 SessionID 的 Cookie。若该用户拔掉网线，让笔记本切换连接到手机热点（即底层网络接口改变，获得了全新的公网 IP 地址），然后再次刷新该电商网站页面。以下关于各层行为的分析中，正确的是（ ）

- A. 传输层的原有 TCP 连接会自动迁移到新 IP 上继续传输，避免连接中断
- B. 应用层 HTTP 协议会发现 IP 地址变更，自动拒绝发送之前保存的 Cookie
- C. 浏览器必须与服务器建立全新的 TCP 连接，但 HTTP 请求中依然会携带原有的 Cookie，购物车状态不会丢失
- D. 服务器由于收到了来自不同 IP 地址的 HTTP 请求，会强制判定为安全风险并立刻销毁该 SessionID 对应的购物车

【答案】：C

【解析】：

传输层/网络层：IP 地址一旦改变，原有的 TCP 连接（由源 IP、目的 IP、源端口、目的端口构成的四元组）就彻底失效了。浏览器必须与服务器进行三次握手，建立全新的 TCP 连接（A 错）

应用层：Cookie 的存储和发送是绑定在域名上的，而不绑定底层 IP 地址。只要用户访问的还是该电商网站的域名，浏览器就会在新的 HTTP 请求报文中自动附带该域名对应的 Cookie（B 错，C 正确）

服务器端：标准 Web 服务器认 Cookie（SessionID）不认 IP。只要 SessionID 合法且未过期，服务器就会读取对应的购物车数据，不会仅因为 IP 变化就销毁数据（除非有极端严格的风控策略，但协议标准不支持 D 选项）

27. 【考点：DNS 系统域名的解析过程】DNS 解析过程中的缓存机制极大地降低了网络负载。假设某台本地域名服务器（Local DNS）刚刚完成了对 mail.cctv.com 的完整递归/迭代解析，并将结果存入缓存（TTL 为 1 天）。紧接着，另一个用户向该本地域名服务器请求解析 [www.cctv.com] (https://www.cctv.com)。针对这第二次查询，在本地域名服务器依然采用迭代查询的前提下，它下一步首先会直接向哪一级的域名服务器发出查询请求报文？（ ）

- A. 根域名服务器
- B. .com 顶级域名服务器
- C. cctv.com 授权域名服务器
- D. 它会直接向用户返回最终 IP 地址，因为缓存已命中

【答案】：C

【解析】：

D 错：之前缓存的是 mail.cctv.com 的映射关系，现在查询的是 [www.cctv.com] (https://www.cctv.com)。这是两个不同的完整主机名 (FQDN)，IP 不一定相同，因此无法直接返回最终 IP

但是，在解析 mail.cctv.com 时，本地域名服务器不仅缓存了最终 IP，还缓存了上级域名服务器的地址，即它已经知道了 cctv.com 授权域名服务器的 IP 地址

因此，当解析同属 cctv.com 域下的新主机名时，本地域名服务器会绕过根服务器和 .com 顶级服务器，直接向 cctv.com 授权域名服务器发起迭代查询（发出查询报文）

28. 【考点：HTTP 与 DNS 跨层数据报封装】主机 A 向主机 B 的 Web 服务器发起一次 HTTP GET 请求，并在同一个 TCP 报文段中收到了一个 200 OK 响应报文（包含极少的 HTML 数据），随后关闭连接。之后主机 A 向本地 DNS 服务器发起一次 UDP DNS 查询，并收到了应答。假设这四个报文（HTTP 请求、HTTP 响应、DNS 查询、DNS 应答）均未发生分片，且均采用标准 IPv4 网络。若仅对比这四个报文在网络层（IP 首部）的关键控制字段，必定完全相同的字段是（）

- A. 生存时间 (TTL)
- B. 协议 (Protocol)
- C. 标识 (Identification)
- D. 首部长度 (IHL)

【答案】：D

【解析】：

A 错 (TTL)：发送报文（如 HTTP GET、DNS 查询）的初始 TTL 由主机 A 的操作系统决定；而接收报文（HTTP 响应、DNS 应答）的 TTL 是由远端服务器设置且经过路由器逐跳递减的，因此两者几乎不可能相同

B 错 (协议 Protocol)：HTTP 使用 TCP (协议号为 6)，DNS 通常使用 UDP (协议号为 17)，四者的 IP 首部协议字段不完全相同

C 错 (标识 Identification)：IP 数据报的标识字段通常是由发送主机的 IP 模块维护的一个计数器，每发送一个数据报递增 1。因此四者的标识字段绝不可能相同

D 正确 (首部长度 IHL)：除非显式使用 IP 选项字段，标准的 IPv4 首部长度固定为 20 字节。IP 首部长度字段（占 4 位，以 4 字节为单位）的值必定固定为 5。无论上层是 TCP 还是 UDP，是请求还是响应，这四个报文的 IP 首部长度字段完全相同

29. (1) DNS 解析总时延计算

主机 H 向本地 DNS 发送查询请求，本地 DNS 最终将结果返回给 H，这属于一次完整的 H 与本地 DNS 之间的交互，消耗 $1 \times RTT_1 = 10 \text{ ms}$

在此期间，本地 DNS 需要分别向根、顶级、授权三个服务器依次发起完整的交互（迭代查询），每次消耗 $RTT_2 = 30 \text{ ms}$ 。共计 $3 \times RTT_2 = 90 \text{ ms}$

总解析时延 = $10 + 3 \times 30 = 100 \text{ ms}$

- (2) HTTP/1.0 协议时延与连接数计算

HTTP/1.0 默认使用非持久连接（短连接），每请求一个对象都必须重新建立 TCP 连接

请求 HTML 主页文件：TCP 三次握手消耗 $1 \times RTT_3$ （第 3 次握手通常携带 HTTP 请求）；服务器返回 HTML 消耗 $1 \times RTT_3$ 。共 $2 \times RTT_3$

请求 4 张图片：每张图片都需要独立的 TCP 连接，均消耗 $2 \times RTT_3$ 。因为题目默认按照典型的串行发起计算，4 张图片共耗时 $4 \times 2 \times RTT_3 = 8 \times RTT_3$

总时间 = $(2 + 8) \times RTT_3 = 10 \times 50 \text{ ms} = 500 \text{ ms}$

TCP 连接数: 1 个 HTML + 4 张图片 = 5 个对象, 因此建立 5 次 TCP 连接

HTTP 交互阶段共消耗 500 ms。主机 H 与 Web 服务器之间总共建立了 5 个 TCP 连接

(3) HTTP/1.1 (非流水线) 时延计算

HTTP/1.1 默认开启持久连接 (Connection: keep-alive)。所有对象可以在同一个 TCP 连接上进行传输

建立 TCP 连接并请求 HTML 主页: 依然消耗 $2 \times RTT_3$

非流水线方式下, 必须在收到前一个对象的响应后, 才能发出下一个对象的请求。4 张图片依次请求, 每张图片由于无需新建 TCP, 仅消耗 $1 \times RTT_3$ 的 HTTP 请求/响应交互时间。4 张图片共消耗 $4 \times RTT_3$

总时间 = $(2 + 4) \times RTT_3 = 6 \times 50 \text{ ms} = 300 \text{ ms}$

HTTP 交互阶段共消耗 300 ms

(4) 端口复用与连接区分机制

Web 服务器提供 HTTP 服务的熟知端口始终是 80 (或 HTTPS 的 443)

Web 服务器使用的本地端口号完全相同 (均为 80 端口)

TCP 协议并非仅通过目的端口来区分连接, 而是通过“TCP 连接的四元组”来唯一标识和区分一条连接。该四元组包含: (源 IP 地址, 目的 IP 地址, 源端口号, 目的端口号)。由于主机 H 每次建立新连接时操作系统分配的源临时端口号不同, 因此 Web 服务器能够依靠源端口号准确区分这 5 个不同的 TCP 连接

30. (1) PORT 指令端口计算

FTP 主动模式下, 客户端通过 PORT 命令告诉服务器自己开放的 IP 和端口, 以便服务器主动连接。PORT 参数格式为 `h1,h2,h3,h4,p1,p2`, 其中前四个数字是 IP 地址, 后两个数字用于计算端口号: 端口号 = $p1 \times 256 + p2$

IP 地址: `192.168.1.100`

端口号计算: $195 \times 256 + 80 = 49920 + 80 = 50000$

FTP 服务器将尝试向 IP `192.168.1.100` 的 50000 端口发起 TCP SYN 数据连接请求

(2) FTP 与 NAT 的跨层冲突与解决

失败原因: 因为 FTP 的 PORT 命令封装在 TCP 报文的数据载荷 (应用层数据) 中, 普通的 NAT 路由器无法识别并修改应用层数据。FTP 服务器收到的依然是私网 IP `192.168.1.100`。由于私网 IP 地址在 Internet 公网上不可路由, FTP 服务器发出的 TCP SYN 报文会被互联网直接丢弃, 导致数据连接建立失败

应用层解决方案: Alice 的 FTP 客户端应该改为使用 被动模式 (Passive/PASV 模式)。在被动模式下, 由客户端主动发起建立数据连接, 从而完美避开外部网络无法主动连接 NAT 内网主机的问题

(3) 电子邮件协议流转

① Alice (发送方用户代理) 到 Alice 邮件服务器: SMTP 协议

② Alice 邮件服务器到 Bob 邮件服务器: SMTP 协议

③ Bob (接收方用户代理) 从 Bob 邮件服务器拉取邮件: POP3 协议 或 IMAP 协议

(4) DNS 与邮件路由

邮件服务器发送邮件时, 根据收件人的邮箱后缀, 通过 DNS 查找负责接收该域邮件的服务器

查询的具体域名: `target-corp.com` (注意: 不是带 `mail` 的主机名, 而是邮箱地址 `@` 后面的邮件域本身)

查询的 DNS 记录类型: MX 记录 (邮件交换记录 / Mail Exchanger Record)